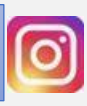
@mohammaed_[redacted]
@its.btec.it







0779538251

دليل شامل لكتابة تقرير يحقق جميع المعايير(P1, P2, P3, M1, D1)

قالب لمهمة البرمجة

روابط جميع قروباتنا ع المواقع التواصفل الاجتماعي وروابط حساباتي الشخصية:

حساب الأنستقرام الخاص بالBTEC:
https://www.instagram.com/its.btec.it?igsh=MXU0YTNzcXl5NXJxMQ==

حسابي الأنستقرام الأساسي:
https://www.instagram.com/mohammaed_atallah?igsh=MXM2bGthempqZGthYw==

رابط قروب الواتس:
https://whatsapp.com/channel/0029VbCGD6F35fLlyUqjhC1w

رابط قروب الأنستقرام:
/https://ig.me/j/AbaGIL8E5ZDzuUSQ

رابط قروب الفيسبوك:
https://www.facebook.com/share/g/1CwFepDd9B/?mibextid=wwXI

محمد عطاالله

المقدمة

نظرة عامة

- شرح موجز عن الهدف من التقرير
- ذكر البرامج النموذجية التي تمت دراستها
- توضيح هيكل التقرير وما سيتم تغطيته
- أهمية التفكير الحسابي في تطوير البرمجيات

P1: التفكير الحسابي

1. تعريف التفكير الحسابي

- ما هو التفكير الحسابي؟ (Computational Thinking)
- أهميته في حل المشكلات البرمجية

العلاقة بين التفكير الحسابي وتطوير البرمجيات

2. المكونات الأربعة للتفكير الحسابي

- تقسيم المشكلة الكبيرة إلى أجزاء أصغر :(Decomposition) التحليل والتفكيك
- مثال عملي من البرنامج النموذجي -
- كيف طبق المبرمج هذا المبدأ -
-
- إيجاد أوجه التشابه :(Pattern Recognition) التعرف على الأنماط
- الأنماط المكررة في الكود -
- أمثلة من البرامج المدروسة -
-
- التركيز على المعلومات المهمة وإهمال التفاصيل غير الضرورية :(Abstraction) التجريد
- كيف تم تجريد البيانات في البرامج -
- كمثال على التجريد Functions/Methods استخدام -
-
- إنشاء خطوات منطقية لحل المشكلة :(Algorithm Design) تصميم الخوارزميات

- الخوارزميات المستخدمة في البرامج النموذجية -
- كأمثلة Pseudocode أو Flowcharts -
-

3. كيف يستخدم المبرمجون التفكير الحسابي

- تحديد المشكلة ووصفها بدقة
- التواصل حول الميزات والعمليات (Documentation, Comments)
- تمثيل المشكلات أو الأنظمة (Diagrams, Models)
- أمثلة محددة من البرامج المقدمة

P2: مبادئ البرمجة في لغات مختلفة

1. لغات البرمجة المدروسة

- قائمة باللغات (مثال: Python, Java, C++, JavaScript)
- نوع كل لغة (Procedural, OOP, Scripting, etc.)
- الخصائص الأساسية لكل لغة

2. البنى البرمجية (Programming Constructs)

- ◈ المتغيرات وأنواع البيانات (Variables & Data Types)
- كيف تختلف بين اللغات المختلفة -
- أمثلة من الكود النموذجي -
-
- ◈ الهياكل الشرطية (Selection: IF, ELSE, SWITCH)
- التنفيذ في لغة 1 -
- التنفيذ في لغة 2 -
- المقارنة بين التنفيذين -
-
- ◈ الحلقات التكرارية (Iteration: FOR, WHILE, DO-WHILE)
- أمثلة من البرامج المختلفة -
- الفروقات في البناء الجملي (Syntax) -
-

محمد عطاالله

- ◈ الدوال/الطرق (Functions/Methods)
- - كيف يتم تعريفها واستدعاؤها
- - Parameters & Return Values
-
- ◈ هياكل البيانات (Arrays, Lists, Dictionaries)
- - التنفيذ في لغات مختلفة
-
- ◈ البرمجة الكائنية (OOP: Classes, Objects, Inheritance)
- - إذا كانت موجودة في البرامج المدروسة

3. تطبيق المبادئ البرمجية

- كيف تم استخدام هذه البنى لحل المشاكل المحددة
- التوثيق (Documentation) والتعليقات (Comments)
- أمثلة ملموسة من الكود المقدم

P3: مبادئ تصميم البرمجيات وجودة التطبيقات

1. مبادئ تصميم البرمجيات

- ◈ التخطيط (Planning): Requirements Analysis
- ◈ التصميم (Design): Architecture, UML Diagrams
- ◈ التنفيذ (Implementation): Coding Standards
- ◈ الاختبار (Testing): Unit Tests, Integration Tests
- ◈ الصيانة (Maintenance): Updates, Bug Fixes

2. خصائص البرمجيات عالية الجودة

- ☑ تلبية متطلبات المستخدم (Meeting User Requirements)
- - هل البرنامج يحقق الأهداف المطلوبة؟
- - أمثلة من البرامج المدروسة
-
- ☑ المتانة (Robustness)
- - معالجة الأخطاء (Error Handling)
- - التحقق من صحة المدخلات (Input Validation)

محمد عطاالله

- ✅ قابلية الصيانة (Maintainability)
- - وضوح الكود (Clean Code)
- - التوثيق الجيد
- - استخدام أسماء متغيرات واضحة
- ✅ الكفاءة (Efficiency)
- - استخدام الموارد (Memory, CPU)
- - تعقيد الخوارزميات (Time Complexity)
- ✅ قابلية النقل (Portability)
- - هل يعمل على منصات مختلفة؟
- ✅ سهولة الاستخدام (Ease of Use)
- - واجهة المستخدم (UI)
- - User Experience (UX)

3. تطبيق المبادئ في البرامج المدروسة

- تحليل برنامج نموذجي محدد
- كيف تم تطبيق مبادئ التصميم
- تقييم مدى تحقيق الجودة

M1: تحليل تأثير التفكير الحسابي

1. تحليل التأثير على التصميم

- 📊 كيف أثر التحليل والتفكيك على بنية البرنامج؟
- - تحليل مفصل مع أمثلة
- - نقاط القوة والضعف
- 📊 تأثير التعرف على الأنماط على إعادة استخدام الكود
- - Functions المشتركة
- - تجنب التكرار (DRY Principle)

محمد عطاالله

-
- تأثير التجريد على وضوح الكود 📊
- - Abstraction Layers
- - Information Hiding
-
- تأثير تصميم الخوارزميات على الكفاءة 📊
- - مقارنة خوارزميات مختلفة
- - Big O Notation (إن أمكن)

2. تحليل تنفيذ اللغات المختلفة

- مقارنة متوازنة بين اللغات المستخدمة
- نقاط القوة والضعف لكل لغة في حل المشكلة
- لماذا تم اختيار لغة معينة لمشكلة محددة؟
- تأثير اختيار اللغة على:
- - سرعة التطوير
- - الأداء
- - قابلية الصيانة

3. تحليل تأثير على جودة البرمجيات

- كيف تحسن التفكير الحسابي من:
- - موثوقية الكود
- - قابلية الصيانة
- - الكفاءة
- أمثلة محددة من البرامج
- تحليل منطقي ومتوازن

D1: تقييم شامل

1. تقييم التفكير الحسابي

- تقييم نقدي لفعالية التفكير الحسابي ☆
- - متى يكون مفيداً؟
- - ما هي حدوده؟
- - أحكام منطقية مدعومة بأدلة

محمد عطاالله

-
- ☆ تقييم كيف أدى التفكير الحسابي إلى:
- تصميم أفضل -
- كود أكثر كفاءة -
- حلول أكثر قابلية للتطوير -

2. تقييم اللغات البرمجية

- تقييم واضح ومتوازن للغات المختلفة
- المزايا التي تقدمها كل لغة للمبرمج والمستخدم النهائي
- لماذا تعتبر لغة معينة "الأفضل" لمشكلة محددة؟
- الحالات التي تكون فيها لغة أفضل من أخرى
- تقييم مبني على معايير واضحة

3. تقييم الجودة الشاملة

- تقييم نقدي لجودة البرامج المدروسة من حيث:
- ✓ درجة استيفاء متطلبات المستخدم
- ✓ متانة الكود
- ✓ قابلية الصيانة
- ✓ الكفاءة
- ✓ قابلية النقل
- ✓ سهولة الاستخدام
-
- أحكام منطقية وصحيحة مدعومة بأدلة
- نظرة شاملة على العلاقة بين التصميم والجودة

4. الاستنتاجات والتوصيات

- أفضل الممارسات المستنتجة
- توصيات لتحسين الجودة
- الدروس المستفادة

إضافة: المنطق الرياضي والبرمجي

دعم إضافي

محمد عطاالله

المنطق في تصميم البرامج

- 🔢 المنطق الرياضي (Mathematical Logic)
- - Boolean Logic
- - Logical Operators (AND, OR, NOT)
-
- 🔢 المنطق القضوي (Propositional Logic)
- - True/False Statements
- - Truth Tables
-
- 🔢 المجموعات (Sets)
- - Set Operations
- - استخدامها في البرمجة
-
- 🔢 التكرار (Iteration)
- - For loops
- - While loops
- - Recursion
-
- أمثلة من البرامج المدروسة

الخاتمة

ملخص نهائي

- تلخيص النقاط الرئيسية
- التأكيد على أهمية التفكير الحسابي
- الربط بين جميع المعايير المغطاة
- نظرة مستقبلية

المراجع

قائمة المصادر

محمد عطاالله

- الكتب المستخدمة (مثل: Stroustrup)
- مواقع إلكترونية موثوقة
- الوثائق الرسمية للغات البرمجة
- البرامج النموذجية المدروسة
- استخدام APA Style أو Harvard

ملاحظات مهمة جداً

📌

الكلمات المفتاحية: Pass (P) = Explain | Merit (M) = Analyse | Distinction (D) = Evaluate

📌

الأمثلة: يجب أن تكون جميع النقاط مدعومة بأمثلة محددة من البرامج النموذجية المقدمة

📌

التقييم (D1): يجب أن يكون نقدياً ومتوازناً مع أحكام منطقية مدعومة بأدلة

📌

الجودة: استخدم مفردات تقنية دقيقة، تنظيم جيد، وربط واضح بين الأفكار

📌

محمد عطاالله

روابط جميع قروباتنا ع المواقع التواصل الاجتماعي وروابط حساباتي الشخصية:

حساب الأنستقرام الخاص بالBTEC:

https://www.instagram.com/its.btec.it?igsh=MXU0YTNzcXl5NXJxMQ==

حسابي الأنستقرام الأساسي:

https://www.instagram.com/mohammaed_atallah?igsh=MXM2bGthempqZGthYw==

رابط قروب الواتس:

https://whatsapp.com/channel/0029VbCGD6F35fLlyUqjhC1w

رابط قروب الأنستقرام:

/https://ig.me/j/AbaGIL8E5ZDzuUSQ

رابط قروب الفيسبوك:

https://www.facebook.com/share/g/1CwFepDd9B/?mibextid=wwXI

محمد عطاالله